ATME COLLEGE OF ENGINEERING

13th KM Stone, Bannur Road, Mysore - 560 028

DEPARTMENT OF CSE (DATA SCIENCE)

LABORATORY REPORT

of

Statistical Machine Learning for Data Science - BAD702

Academic Year 2026-27

Submitted by

Name:

USN:

Faculty Incharge

Dr. Anitha D B

Associate Professor and HoD

CSE (Data Science)

ATME COLLEGE OF ENGINEERING

13th KM Stone, Bannur Road, Mysore - 560 028

DEPARTMENT OF CSE (DATA SCIENCE)

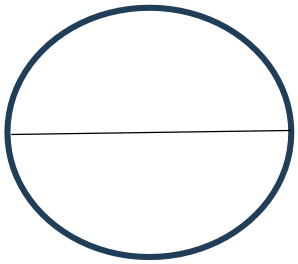
CERTIFICATE

This is to certify that Mr./ Mrs ……

Bearing University Seat Number …….

has satisfactorily completed the laboratory Experiments in the course titled……

…….in the ……Semester of B.E Program during the academic year 2026-27 as prescribed by the Visveswaraya Technological University Belagavi.



Signature of the Staff

Signature of the HOD

INDEX

Sl No.	Experiments	Date	Page No	Marks
1	A dataset contains the prices of houses in a city. Find the 25th and 75th percentiles and calculate the interquartile range (IQR). How does the IQR help in understanding the price variability?			
2	You are given a dataset with categorical variables about customer satisfaction levels (Low, Medium, High) and whether customers made repeat purchases (Yes/No). Create visualizations such as bar plots or stacked bar charts to explore the relationship between satisfaction level and repeat purchases. What can you infer from the data?			
3	A dataset contains information about car models, including the engine size (in Liters), fuel efficiency (miles per gallon), and car price. Use a pair plot or correlation matrix to explore the relationships between these variables. Which variables seem to have the strongest relationships, and what might be the practical significance of these findings?			
4	You want to estimate the mean salary of software engineers in a country. You take 10 different random samples, each containing 50 engineers, and calculate the sample mean for each. Plot the distribution of these sample means. How does the Central Limit Theorem explain the shape of this sampling distribution, even if the underlying salary distribution is skewed?			
5	A researcher conducts an experiment with a sample of 20 participants to determine if a new drug affects heart rate. The			

	sample has a mean heart rate increase of 8 beats per minute and a standard deviation of 2 beats per minute. Perform a hypothesis test using the t-distribution to determine if the mean heart rate increase is significantly different from zero at the 5% significance level.			
6	A company is testing two versions of a webpage (A and B) to determine which version leads to more sales. Version A was shown to 1,000 users and resulted in 120 sales. Version B was shown to 1,200 users and resulted in 150 sales. Perform an A/B test to determine if there is a statistically significant difference in the conversion rates between the two versions. Use a 5% significance level			
7	You are comparing the average daily sales between two stores. Store A has a mean daily sales value of $1,000 with a standard deviation of $100 over 30 days, and Store B has a mean daily sales value of $950 with a standard deviation of $120 over 30 days. Conduct a two-sample t-test to determine if there is a significant difference between the average sales of the two stores at the 5% significance level.			
8	A company collects data on employees' salaries and records their education level as a categorical variable with three levels: "High School", "Bachelor's", and "Master's". Fit a multiple linear regression model to predict salary using education level (as a factor variable) and years of experience. Interpret the coefficients for the education levels in the regression model.			
9	You have data on housing prices and square footage and notice that the relationship between square footage and price is nonlinear. Fit a spline regression model to allow the relationship between square footage and price to change at 2,000 square feet. Explain how spline regression can capture			

	different behaviors of the relationship before and after 2,000 square feet.			
10	A hospital is using a Poisson regression model (a type of GLM) to predict the number of emergency room visits per week based on patient age and medical history. The model is given by: $\text{Log}(\lambda) = 2.5 - 0.03*\text{Age} + 0.5*\text{condition}$ where λ is the expected number of visits per week, Age is the patient's age, and condition is a binary variable (1 if the patient has a chronic condition, 0 otherwise). Interpret the coefficients of Age and condition. What is the expected number of visits per week for a 60-year-old patient with a chronic condition? How would the expected number of visits change if the patient did not have a chronic condition?			
11	A bakery claims that its new cookie recipe is lower in calories compared to the old recipe, which had a mean calorie count of 200. You sample 40 new cookies and find a mean of 190 calories with a standard deviation of 15 calories. Perform a one-tailed t-test to determine if the new recipe has significantly fewer calories at a 5% significance level.			
Average:				

Total Marks: _____ + _____ = _____

15 (LCR) **10 (TEST)** **25**

Signature of Staff

EXPERIMENT 1

A dataset contains the prices of houses in a city. Find the 25th and 75th percentiles and calculate the interquartile range (IQR). How does the IQR help in understanding the price variability?

The Interquartile Range (IQR) tells us how spread out the middle 50% of the data is. It is less affected by extreme values and gives a better idea of how tightly or loosely the central data points are grouped. It is calculated using the first quartile (Q1) and third quartile (Q3).

- Focuses only on the central portion of the data.
- Not affected much by outliers, unlike the full range.
- Gives a clear view of how values are arranged around the median.

Quartiles divide the dataset into four equal parts:

- **Q1 (First Quartile):** Separates the lowest 25% of the data
- **Q2 (Second Quartile / Median):** Middle value of the dataset
- **Q3 (Third Quartile):** Separates the highest 25% of the data

These three cut points split the dataset into four groups:

1. From the minimum value to Q1 (lowest 25%)
2. From Q1 to Q2 (next 25%)
3. From Q2 to Q3 (next 25%)
4. From Q3 to the maximum value (highest 25%)

Ordered Data (Smallest → Largest)

```
Min       Q1          Q2 (Median)       Q3          Max
 |----------|--------------|--------------|------------|
     25%         25%            25%          25%
```

The IQR captures the range between Q1 and Q3, representing the middle 50% of the distribution.

Formula of IQR

The data is sorted in ascending order and split into 4 equal parts: Q_1, Q_2, Q_3, called first, second, and third quartiles, respectively, in the given data.

The IQR is simply the difference between the third quartile (Q3) and the first quartile (Q1), which is calculated as: **Interquartile Range = Upper Quartile (Q3)– Lower Quartile(Q1)**

It tells us how spread out the central 50% of the data is, which helps to gauge the data's variability

without being influenced by outliers.

How to Calculate the Interquartile Range

Step 1: Sort the data in ascending order

Step 2: Find the median (Q2)

Step 3: Split the dataset into two halves

- Q1 = median of lower half
- Q3 = median of upper half

Step 4: Calculate IQR

IQR = Q3 - Q1

Example

Dataset: 77, 85, 92, 64, 78, 95, 82

Sort the data: 64, 77, 78, 82, 85, 92, 95

Median (Q2) = 82

Lower half = 64, 77, 78

- Q1 is the median of the lower half (64, 77, 78) which is 77.

Upper half = 85, 92, 95

- Q3 is the median of the upper half (85, 92, 95) which is 92.

Program:

#import necessary libraries

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import numpy as np

data=pd.read_csv('House Price India.csv')

data.head()
```

Output:

	id	Date	number of bedrooms	number of bathrooms	living area	lot area	number of floors	waterfront present	number of views	condition of the house	...	Built Year	Renovation Year
0	6762810635	42491	4	2.50	2920	4000	1.5	0	0	5	...	1909	0
1	6762810998	42491	5	2.75	2910	9480	1.5	0	0	3	...	1939	0
2	6762812605	42491	4	2.50	3310	42998	2.0	0	0	3	...	2001	0
3	6762812919	42491	3	2.00	2710	4500	1.5	0	0	4	...	1929	0
4	6762813105	42491	3	2.50	2600	4750	1.0	0	0	4	...	1951	0

5 rows × 23 columns

Postal Code	Lattitude	Longitude	living_area_renov	lot_area_renov	Number of schools nearby	Distance from the airport	Price
122004	52.8878	-114.470	2470	4000	2	51	1400000
122004	52.8852	-114.468	2940	6600	1	53	1200000
122005	52.9532	-114.321	3350	42847	3	76	838000
122006	52.9047	-114.485	2060	4500	1	51	805000
122007	52.9133	-114.590	2380	4750	1	67	790000

```
print(data.columns)
```

Output:

```
Index(['id', 'Date', 'number of bedrooms', 'number of bathrooms',
       'living area', 'lot area', 'number of floors', 'waterfront present',
       'number of views', 'condition of the house', 'grade of the house',
       'Area of the house(excluding basement)', 'Area of the basement',
       'Built Year', 'Renovation Year', 'Postal Code', 'Lattitude',
       'Longitude', 'living_area_renov', 'lot_area_renov',
       'Number of schools nearby', 'Distance from the airport', 'Price'],
      dtype='object')
```

```
data.isnull().sum()
```

Output:

```
id                                      0
Date                                    0
number of bedrooms                      0
number of bathrooms                     0
living area                             0
lot area                                0
number of floors                        0
waterfront present                      0
number of views                         0
condition of the house                  0
grade of the house                      0
Area of the house(excluding basement)   0
Area of the basement                    0
Built Year                              0
Renovation Year                         0
Postal Code                             0
Lattitude                               0
Longitude                               0
living_area_renov                       0
lot_area_renov                          0
Number of schools nearby                0
Distance from the airport               0
Price                                   0
dtype: int64
```

Method 1

```
Q1=np.percentile(data['Price'],25)
Q3=np.percentile(data['Price'],75)
IQR=Q3-Q1
print(f"The 25th percentile(Q1) of house price is:${Q1:,.2f}")
print(f"The 75th percentile (Q3) of house price is:${Q3:,.2f}")
print(f"The interquartile (IQR) of house price is:${IQR:,.2f}")
```

Output:

```
The 25th percentile(Q1) of house price is:$320,000.00
The 75th percentile (Q3) of house price is:$645,000.00
The interquartile (IQR) of house price is:$325,000.00
```

Method 2

```
Q1=data['Price'].quantile(0.25)
Q3=data['Price'].quantile(0.75)
IQR=Q3-Q1
print("The 25th percentile(Q1) of house price is:",Q1)
print("The 75th percentile (Q3) of house price is:",Q3)
print("The interquartile (IQR) of house price is:",IQR)
```

Output:

The 25th percentile(Q1) of house price is: 320000.0

The 75th percentile (Q3) of house price is: 645000.0

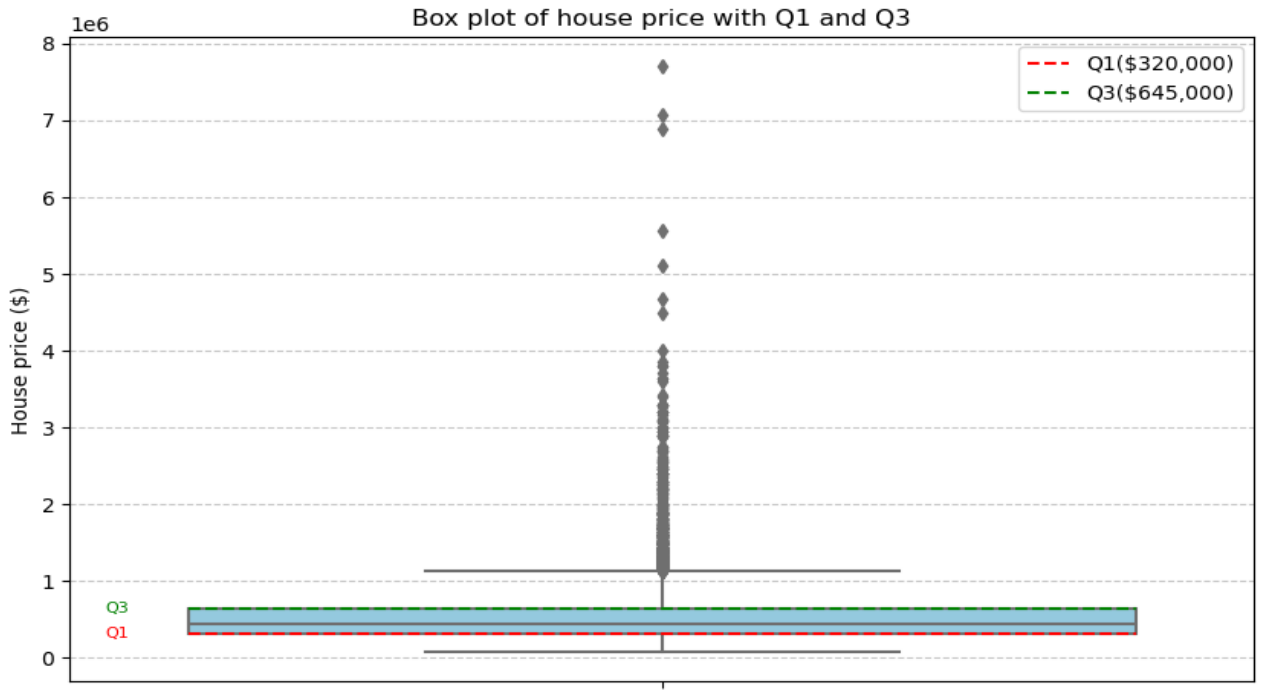
The interquartile (IQR) of house price is: 325000.0

Plot is Optional

```
plt.figure(figsize=(10,6))
sns.boxplot(y=data["Price"],color='skyblue')
plt.hlines(Q1,xmin=-0.4,xmax=0.4,colors='red',linestyles='dashed',label=f'Q1(${Q1:,.0f})')
plt.hlines(Q3,xmin=-0.4,xmax=0.4,colors='green',linestyles='dashed',label=f'Q3(${Q3:,.0f})')
plt.text(-0.45,Q1,'Q1',va='center',ha='right',color='red',fontsize=8)
plt.text(-0.45,Q3,'Q3',va='center',ha='right',color='green',fontsize=8)
plt.title("Box plot of house price with Q1 and Q3")

plt.ylabel("House price ($)")
plt.grid(axis='y',linestyle="--",alpha=0.7)
plt.legend()
plt.show()
```

Output:



Outcome of the Experiment:

EXPERIMENT 2

You are given a dataset with categorical variables about customer satisfaction levels (Low, Medium, High) and whether customers made repeat purchases (Yes/No). Create visualizations such as bar plots or stacked bar charts to explore the relationship between satisfaction level and repeat purchases. What can you infer from the data?

Customer Satisfaction and Repeat Purchase Analysis Using Bar Plot and Stacked Bar Chart

Data visualization is an important part of **Exploratory Data Analysis (EDA)**. It helps in understanding patterns, trends, and relationships within a dataset before applying advanced statistical or machine learning techniques. In this experiment, categorical variables such as **Customer Satisfaction** and **Repeat Purchase** are analyzed using **bar plots** and **stacked bar charts**.

Customer Satisfaction is a measure of how satisfied customers are with a product or service. In this analysis, satisfaction scores are grouped into three categories:

- **Low**
- **Medium**
- **High**

The **Repeat Purchase** variable indicates whether a customer has purchased again from the company. It has two categories:

- **Yes**
- **No**

Since both variables are categorical, graphical methods such as **bar plots** and **stacked bar charts** are suitable for visualizing their relationship.

Bar Plot

A **bar plot** is used to represent the frequency or count of different categories. Each bar corresponds to a category, and the height of the bar represents the number of observations in that category.

In this experiment, the bar plot displays the number of customers in each satisfaction category, with different colors representing customers who made repeat purchases and those who did not. This helps compare the distribution of repeat purchases across different satisfaction levels.

Stacked Bar Chart

A **stacked bar chart** is an extension of the bar plot in which each bar is divided into segments representing different categories of another variable. It is useful for comparing both the total count and the composition

of categories.

In this analysis, each satisfaction level is represented by one bar, which is divided into two segments:

- Customers who made repeat purchases (**Yes**)
- Customers who did not make repeat purchases (**No**)

The stacked bar chart provides a clear view of the relationship between customer satisfaction and repeat purchasing behavior.

Data Preprocessing

The original dataset contains a numerical **customer_satisfaction** score. To perform categorical analysis, the scores are converted into three categories:

- **Low**
- **Medium**
- **High**

Similarly, the **Repeat Purchase** variable is created based on the number of purchases. Customers with purchase counts above a chosen threshold (such as the median) are labeled **Yes**, while the remaining customers are labeled **No**.

Python Libraries Used

- **Pandas:** Used for loading, cleaning, and manipulating the dataset.
- **Matplotlib:** Used for creating plots and customizing visualizations.
- **Seaborn:** Used for creating attractive statistical graphics such as count plots.

Objective

The objective of this experiment is to:

- Analyze the relationship between customer satisfaction and repeat purchases.
- Visualize categorical data using bar plots and stacked bar charts.
- Understand how customer satisfaction influences customer loyalty and repeat buying behavior.

Inference

From the visualizations, it can be observed that customers with **higher satisfaction levels are generally more likely to make repeat purchases**, whereas customers with **lower satisfaction levels are less likely to purchase again**. This indicates a positive relationship between customer satisfaction and customer retention, highlighting the importance of maintaining high customer satisfaction to encourage repeat business.

Program:

```
#import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

#load dataset
df = pd.read_csv("ecommerce_customer_data.csv")
df['Satisfaction Level'] = pd.cut(df['customer_satisfaction'],bins=[1, 2, 3, 5],labels=['Low', 'Medium', 'High'])

# Create Strong Relationship with Repeat Purchase
# High satisfaction → mostly repeat customers
high = df['Satisfaction Level'] == 'High'
df.loc[high, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=high.sum(), p=[0.85, 0.15])

# Medium satisfaction → balanced
medium = df['Satisfaction Level'] == 'Medium'
df.loc[medium, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=medium.sum(), p=[0.55, 0.45])

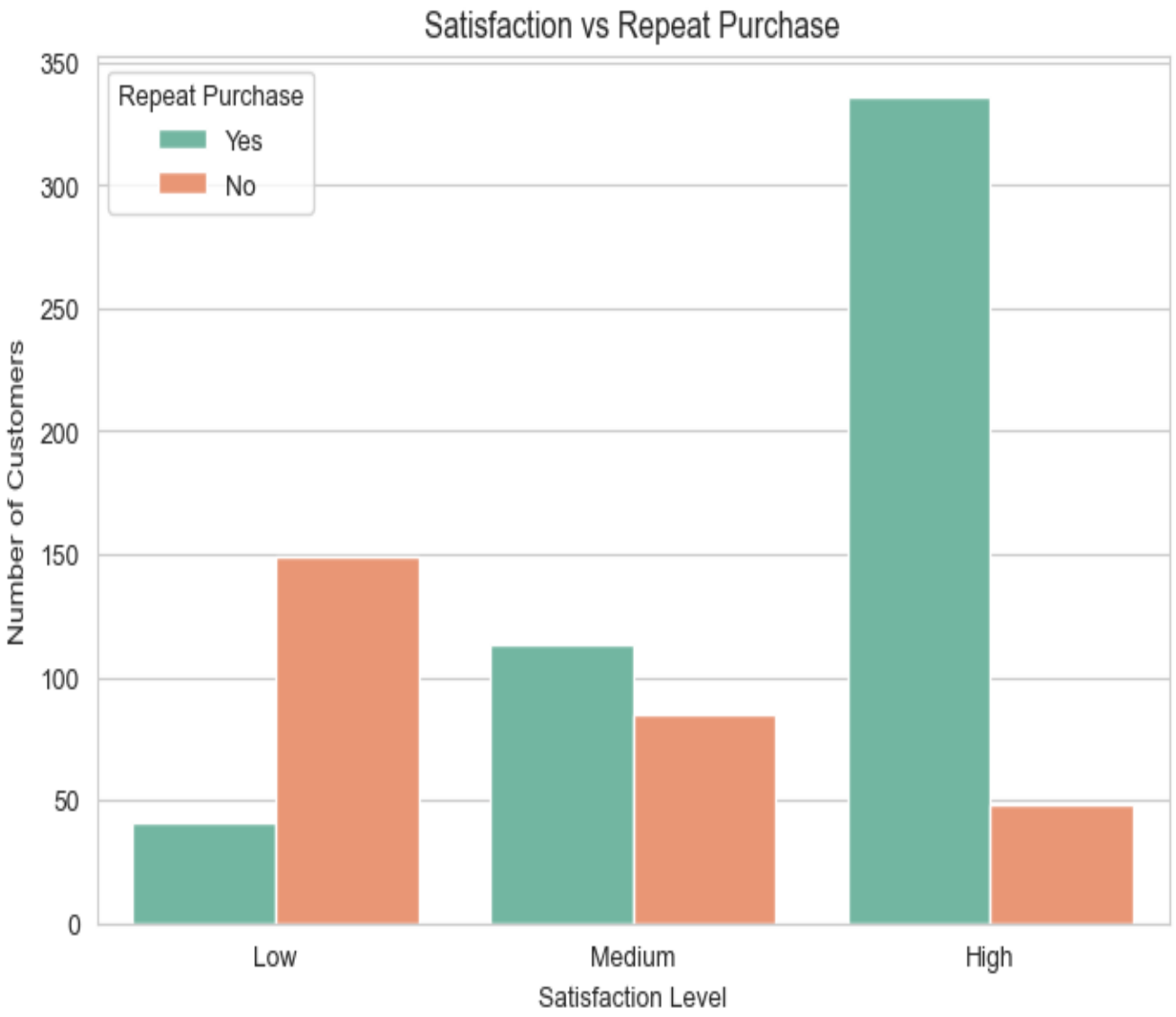
# Low satisfaction → mostly churn
low = df['Satisfaction Level'] == 'Low'
df.loc[low, 'Repeat Purchase'] = np.random.choice(['Yes', 'No'], size=low.sum(), p=[0.25, 0.75])

# Bar Plot
sns.set_style("whitegrid")
plt.figure(figsize=(8,5))
sns.countplot(data=df,x='Satisfaction Level',hue='Repeat Purchase',palette='Set2')

plt.title("Satisfaction vs Repeat Purchase")
plt.xlabel("Satisfaction Level")
plt.ylabel("Number of Customers")
```

```
plt.show()
```

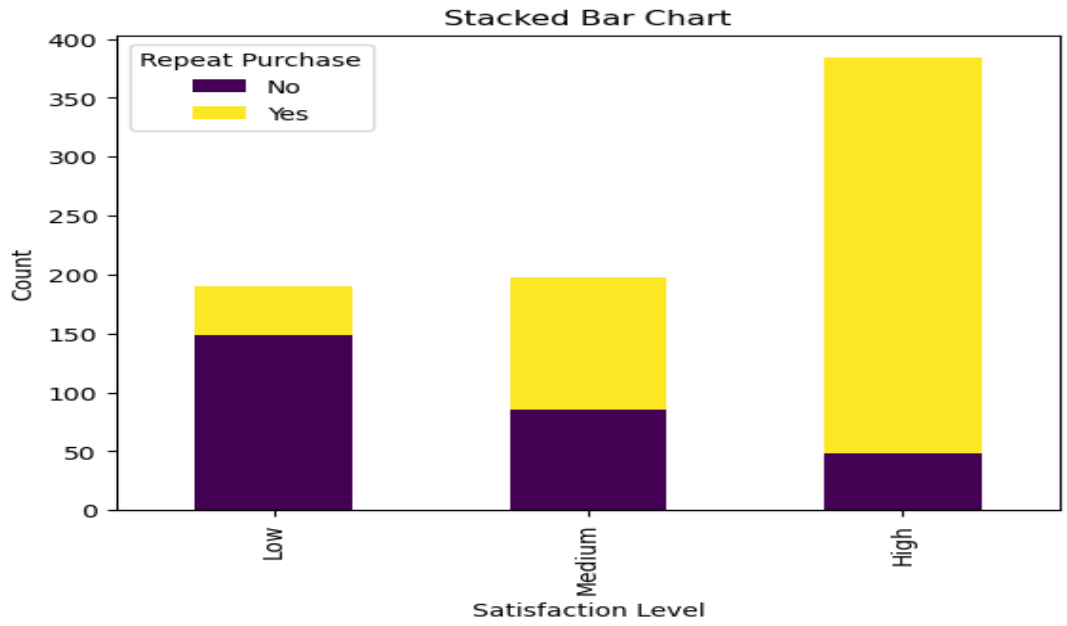
Output:




```
cross = pd.crosstab(df['Satisfaction Level'], df['Repeat Purchase'])
cross.plot(kind='bar', stacked=True, colormap='viridis')

plt.title("Stacked Bar Chart")
plt.ylabel("Count")
plt.show()
print("\nCrosstab:")
print(cross)
```

Output:



Crosstab:

Repeat Purchase	No	Yes
Satisfaction Level		
Low	149	41
Medium	85	113
High	48	336

Percentage Table

```
percentage = pd.crosstab(df['Satisfaction Level'], df['Repeat Purchase'],normalize='index') * 100
print("\nPercentage Distribution:")
print(percentage.round(2))
```

Output:

Percentage Distribution:

Repeat Purchase	No	Yes
Satisfaction Level		
Low	78.42	21.58
Medium	42.93	57.07
High	12.50	87.50

Outcome of the Experiment:

Experiment-3

A dataset contains information about car models, including the engine size (in Liters), fuel efficiency (miles per gallon), and car price. Use a pair plot or correlation matrix to explore the relationships between these variables. Which variables seem to have the strongest relationships, and what might be the practical significance of these findings?

Correlation Matrix

A **correlation matrix** is a table showing correlation coefficients between variables. It helps in understanding how strongly features are related to each other.

Types of Correlation

- **Positive Correlation (+1 to 0)**: As one feature increases, the other also increases.
- **Negative Correlation (0 to -1)**: As one feature increases, the other decreases.
- **No Correlation (0)**: No linear relationship between the variables.

Why Should You Use a Correlation Matrix?

- Identifies relationships between features.
- Helps in detecting multicollinearity in machine learning models.
- Highlights redundant features that may not add value to the model.

In data analysis and machine learning, understanding the relationships between features is crucial for feature selection, multicollinearity detection, and data interpretation. Correlation and pair plots are two essential techniques to analyze these relationships.

1. Correlation Matrix

A **correlation matrix** is a table showing correlation coefficients between variables. It helps in understanding how strongly features are related to each other.

Types of Correlation

- **Positive Correlation (+1 to 0)**: As one feature increases, the other also increases.
- **Negative Correlation (0 to -1)**: As one feature increases, the other decreases.
- **No Correlation (0)**: No linear relationship between the variables.

Why Should You Use a Correlation Matrix?

- Identifies relationships between features.
- Helps in detecting multicollinearity in machine learning models.
- Highlights redundant features that may not add value to the model.

Heatmap for Correlation Matrix:

A **heatmap** is a visual representation of the correlation matrix. It uses color coding to indicate the strength of relationships between variables.

Benefits of Using a Heatmap

- Easy to interpret relationships between features.
- Quickly identifies highly correlated variables.
- Helps in feature selection and data preprocessing

3.Pair Plot

A **pair plot** (also known as a scatterplot matrix) is a collection of scatter plots for every pair of numerical variables in the dataset. It helps in visualizing relationships between variables.

Why Use a Pair Plot?

- Shows the distribution of individual features along the diagonal.
- Displays relationships between features using scatter plots.
- Helps in identifying clusters, trends, and potential outliers.

Summary Statistics Explanation:

The summary statistics table provides key percentiles and other descriptive metrics for each numerical feature:

-**25% (First Quartile - Q1):** This represents the value below which 25% of the data falls.

It helps in understanding the lower bound of typical data values.

50% (Median - Q2): This is the middle value when the data is sorted.

- It provides the central tendency of the dataset.

75% (Third Quartile - Q3): This represents the value below which 75% of the data falls.

- It helps in identifying the upper bound of typical values in the dataset.

These percentiles are useful for detecting skewness, data distribution, and identifying potential outliers (values beyond Q1 - 1.5*IQR or Q3 + 1.5*IQR).

Program:

```
# Importing the required libraries
import pandas as pd
import numpy as np
import seaborn as sns                    #visualisation
import matplotlib.pyplot as plt          #visualisation

# Loading the CSV file into a pandas dataframe.
df = pd.read_csv("Automobile_data.csv")
df.head(5)
```

Output:

	index	company	body-style	wheel-base	length	engine-type	num-of-cylinders	horsepower	average-mileage	price
0	0	alfa-romero	convertible	88.6	168.8	dohc	four	111	21	13495.0
1	1	alfa-romero	convertible	88.6	168.8	dohc	four	111	21	16500.0
2	2	alfa-romero	hatchback	94.5	171.2	ohcv	six	154	19	16500.0
3	3	audi	sedan	99.8	176.6	ohc	four	102	24	13950.0
4	4	audi	sedan	99.4	176.6	ohc	five	115	18	17450.0

```
df.columns()
```

Output:

```
Index(['index', 'company', 'body-style', 'wheel-base', 'length', 'engine-type',
       'num-of-cylinders', 'horsepower', 'average-mileage', 'price'],
      dtype='object')
```

Removing irrelevant features

```
df = df.drop(['index','body-style','engine-type', 'num-of-cylinders'],axis=1)
df.head(5)
```

	company	wheel-base	length	horsepower	average-mileage	price
0	alfa-romero	88.6	168.8	111	21	13495.0
1	alfa-romero	88.6	168.8	111	21	16500.0
2	alfa-romero	94.5	171.2	154	19	16500.0
3	audi	99.8	176.6	102	24	13950.0
4	audi	99.4	176.6	115	18	17450.0

```
df.isnull().sum()
```

Output:

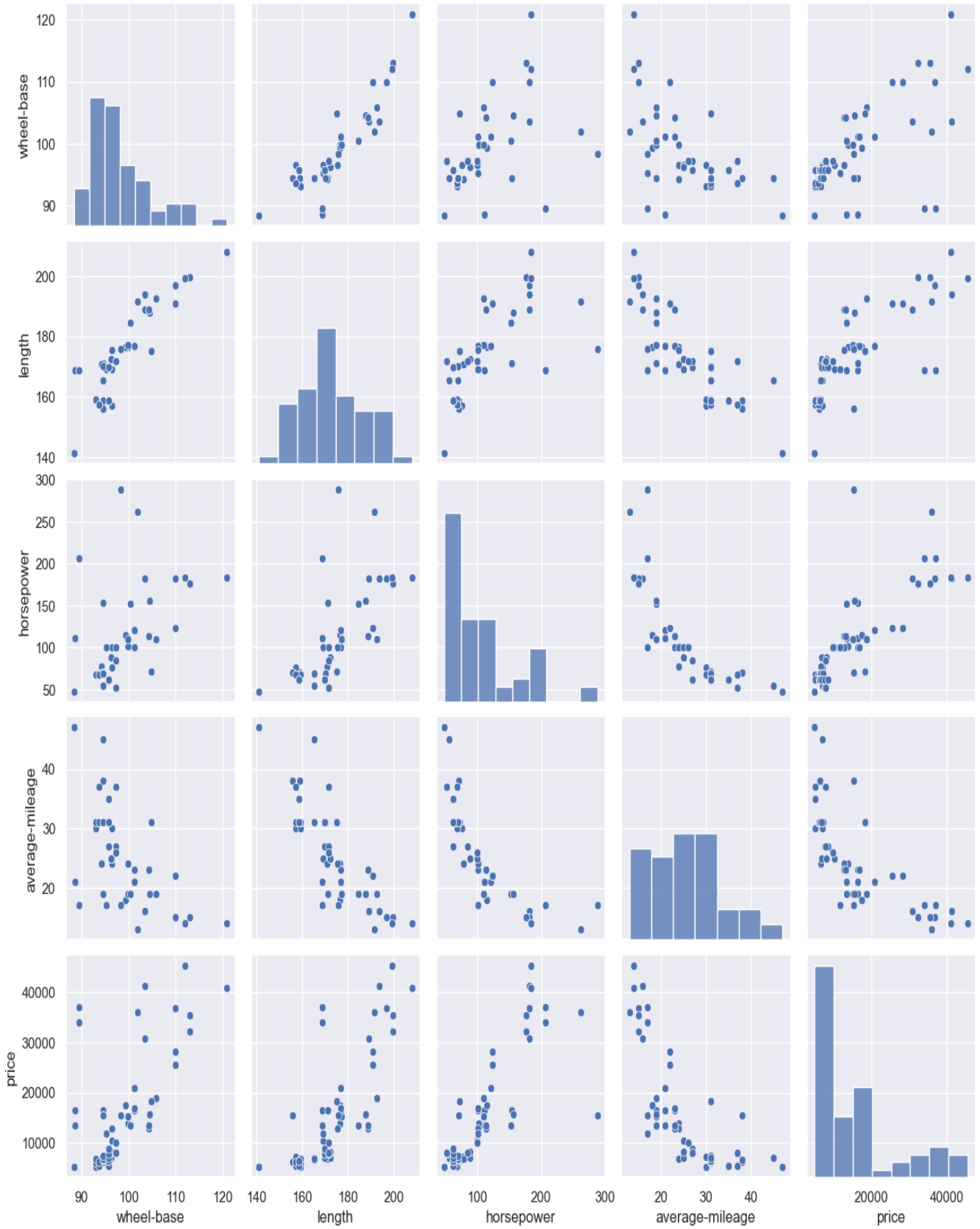
```
company          0
wheel-base       0
length           0
horsepower       0
average-mileage  0
price            3
dtype: int64
```

```
df['price']=df['price'].fillna(df['price'].mean())
```

Generate Pair plot

```
sns.pairplot(df,diag_kind='hist')
plt.show()
```

Output:




```
corr_matrix= df.corr(numeric_only=True)
print("Correlation matrix")
```

print(corr_matrix)

Output:

```
Correlation matrix
                wheel-base    length  horsepower  average-mileage     price
wheel-base        1.000000  0.878381    0.463421        -0.547325  0.659005
length            0.878381  1.000000    0.668555        -0.788429  0.767611
horsepower        0.463421  0.668555    1.000000        -0.808804  0.803195
average-mileage  -0.547325 -0.788429   -0.808804         1.000000 -0.732467
price             0.659005  0.767611    0.803195        -0.732467  1.000000
```

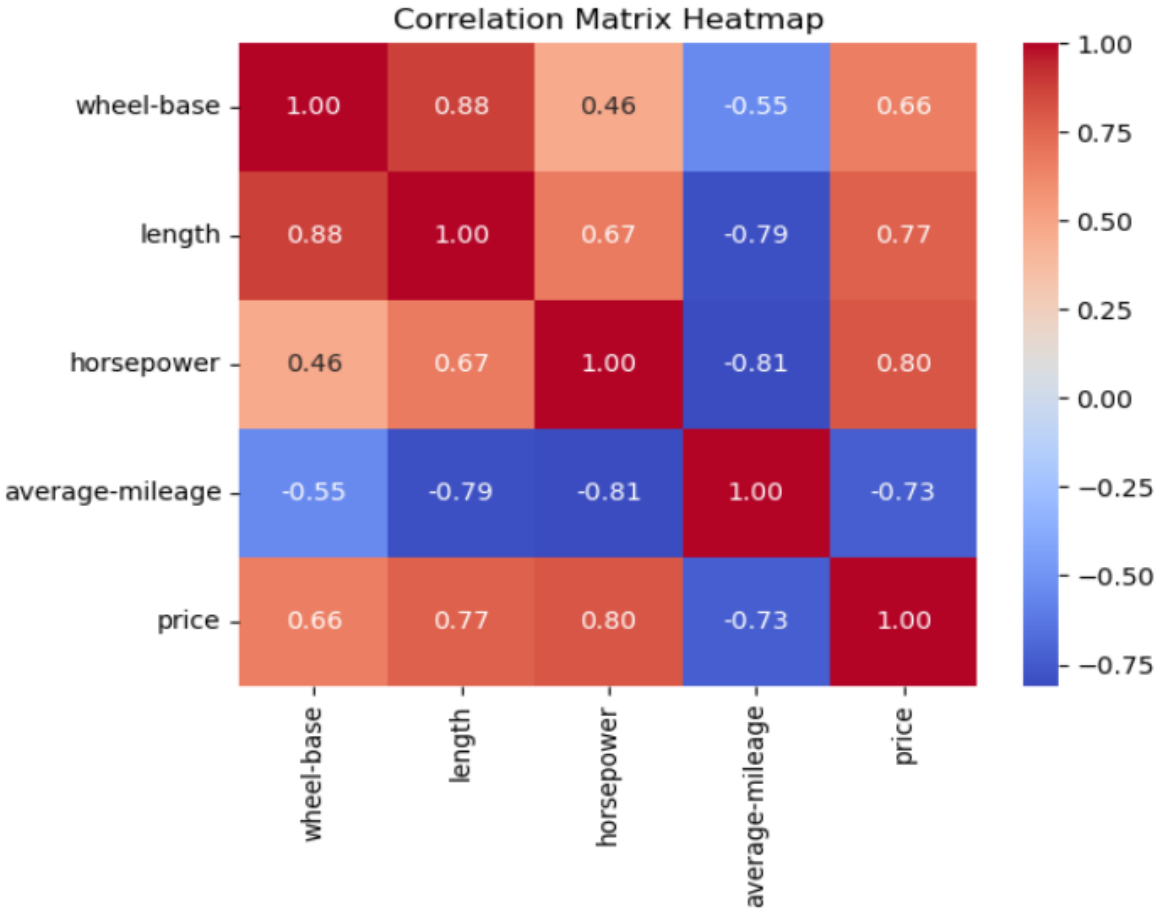
Plotting a heat map

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")

plt.title('Correlation Matrix Heatmap')

plt.show()

Output:



Outcome of the Experiment:

EXPERIMENT 4

You want to estimate the mean salary of software engineers in a country. You take 10 different random samples, each containing 50 engineers, and calculate the sample mean for each. Plot the distribution of these sample means. How does the Central Limit Theorem explain the shape of this sampling distribution, even if the underlying salary distribution is skewed?

Key Terms

- **Population vs. Sample:** In statistics, a *population* includes every element of interest in a study, while a *sample* is a smaller subset of this population.
- **Random Sampling**: In a random sample, each member of the population has an equal chance of being chosen, which helps ensure that the sample is representative.
- **Probability Distribution**: This represents the possible outcomes of a random variable (like the result of a die roll).
- **Sampling Distribution**: A sampling distribution is the probability distribution of a sample statistic, such as the sample mean, based on repeated sampling.

What is the Central Limit Theorem?

The Central Limit Theorem, or CLT, has been foundational in statistics for over 275 years. It states that the *sampling distribution of the mean* will approach a normal (bell-curve) distribution as the sample size increases. In other words, as the sample size grows, the distribution of the sample means will increasingly resemble a normal distribution.

he central limit theorem holds true for any population. You don't need to know the shape of your population distribution in advance to apply the theorem — the distribution could be bell-shaped, skewed, or have another shape. If you collect enough samples of sufficient size, the shape of the distribution of their means will follow a normal distribution. This allows us to estimate population parameters, like the mean, with just a sample.

Example: Estimating the Average Household Income

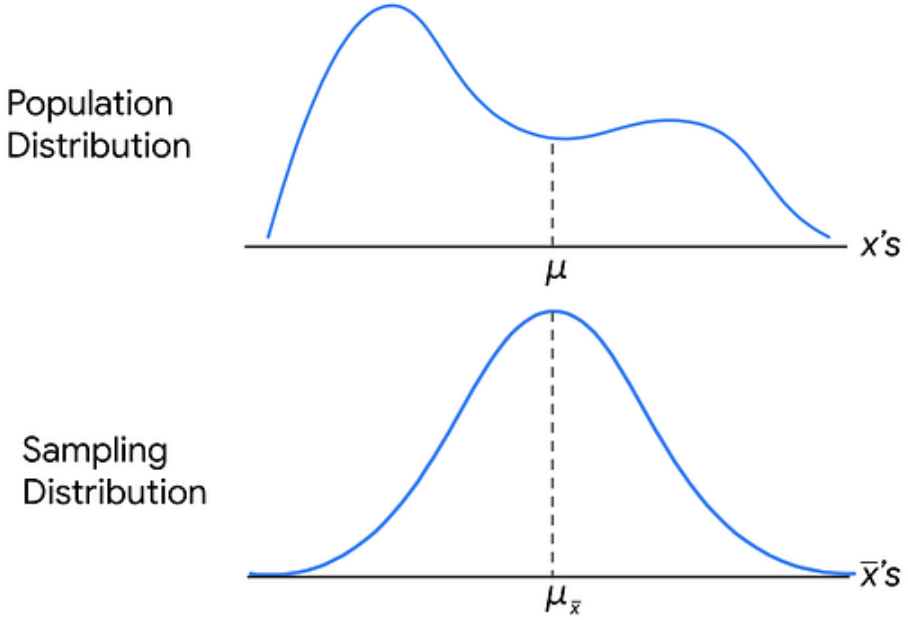
Imagine you want to estimate the average household income in the U.S. Instead of collecting data from millions of households, you can use a representative sample.

1. You take repeated samples from the population (each sample with the same size).
2. For each sample, you calculate the mean household income.

3. The distribution of these sample means forms what's called a *sampling distribution*.

According to the Central Limit Theorem, this sampling distribution will follow a normal distribution, and the average of these sample means will be approximately equal to the population mean.

***Note*:** In practice, due to limited time and resources, data professionals typically collect a single sample (large enough) and calculate the mean of that sample to estimate the population mean**.**



Conditions for Applying the CLT

To accurately apply the CLT, certain conditions must be met:

Randomization: Sample data must be randomly selected from the population.

Independence: Each sample value should be independent of others. Typically, this means the sample should be no larger than 10% of the population when sampling without replacement

Sample Size: A larger sample size improves the accuracy of the approximation.

How large should the sample be?

There's no strict rule, but sample size depends on:

- **Precision requirements**: Larger samples yield more precise estimates.
- **Shape of the population**: If the population is approximately normal, even a smaller sample may be sufficient.

Program:

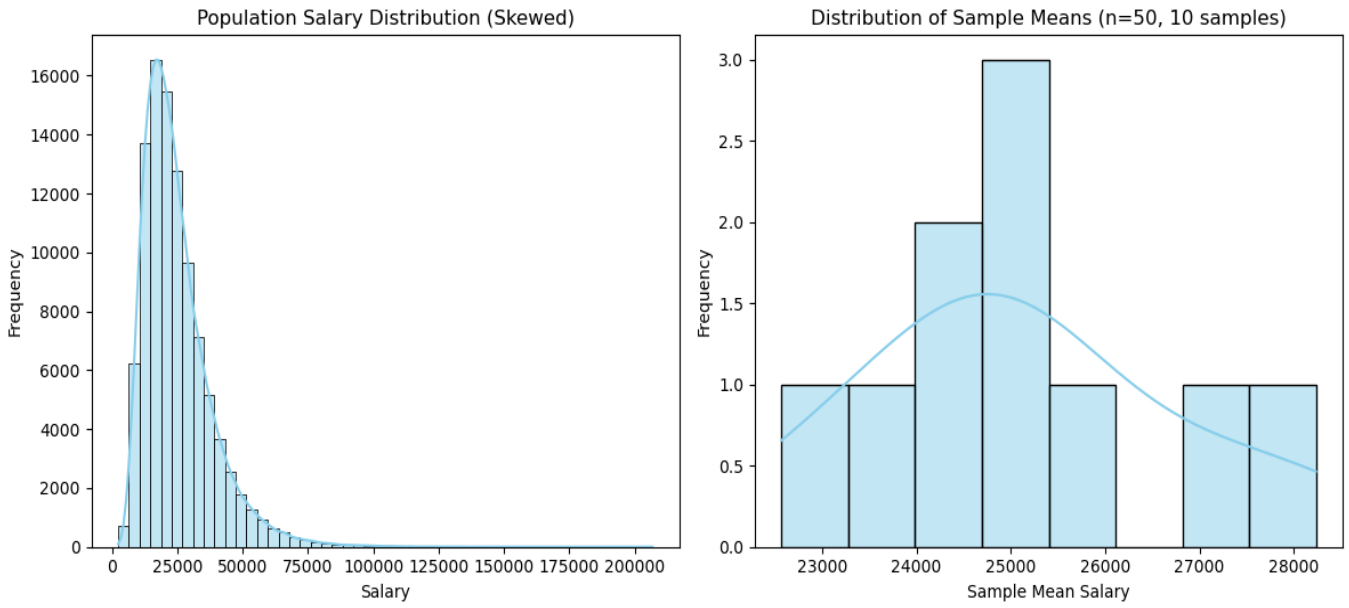
```
#import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
population_size = 100000
population_salaries = np.random.lognormal(mean=10, sigma=0.5, size=population_size)
sample_size = 50
num_samples = 10
sample_means = []
for _ in range(num_samples):
sample = np.random.choice(population_salaries, size=sample_size, replace=False)
sample_means.append(np.mean(sample))

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
#plt.hist(population_salaries,bins=50, color='skyblue', edgecolor='black')
sns.histplot(population_salaries, kde=True, bins=50, color='skyblue', edgecolor='black')
plt.title("Population Salary Distribution (Skewed)")
plt.xlabel("Salary")
plt.ylabel("Frequency")

plt.subplot(1,2,2)
#plt.hist(sample_means, bins=5, color='lightgreen', edgecolor='black')
sns.histplot(sample_means, kde=True, bins=8, color='skyblue', edgecolor='black')
plt.title("Distribution of Sample Means (n=50, 10 samples)")
plt.xlabel("Sample Mean Salary")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

Output:



print("Population Mean Salary:", np.mean(population_salaries))

print("Sample Means:", sample_means)

print("Mean of Sample Means:", np.mean(sample_means))

Output:

Population Mean Salary: 24975.33114237901

Sample Means: [25991.76371315867, 23442.831482490306, 25212.640486412376, 24746.59494962444, 24623.799515214003, 25088.99118838132, 24076.15129376901, 22571.73235048917, 28232.854819077, 27159.490013655606]

Mean of Sample Means: 25114.68498122719

Outcome of the Experiment:

EXPERIMENT 5

A researcher conducts an experiment with a sample of 20 participants to determine if a new drug affects heart rate. The sample has a mean heart rate increase of 8 beats per minute and a standard deviation of 2 beats per minute. Perform a hypothesis test using the t-distribution to determine if the mean heart rate increase is significantly different from zero at the 5% significance level.

Critical values of *t* for one-tailed tests

Significance level (α)

Degrees of freedom (df)	.2	.15	.1	.05	.025	.01	.005	.001
1	1.376	1.963	3.079	6.314	12.706	31.821	63.657	318.309
2	1.061	1.386	1.886	2.920	4.303	6.965	9.925	22.327
3	0.978	1.250	1.638	2.353	3.182	4.541	5.841	10.215
4	0.941	1.190	1.533	2.132	2.776	3.747	4.604	7.173
5	0.920	1.156	1.476	2.015	2.571	3.365	4.032	5.893
6	0.906	1.134	1.440	1.943	2.447	3.143	3.707	5.208
7	0.896	1.119	1.415	1.895	2.365	2.998	3.499	4.785
8	0.889	1.108	1.397	1.860	2.306	2.896	3.355	4.501
9	0.883	1.100	1.383	1.833	2.262	2.821	3.250	4.297
10	0.879	1.093	1.372	1.812	2.228	2.764	3.169	4.144
11	0.876	1.088	1.363	1.796	2.201	2.718	3.106	4.025
12	0.873	1.083	1.356	1.782	2.179	2.681	3.055	3.930
13	0.870	1.079	1.350	1.771	2.160	2.650	3.012	3.852
14	0.868	1.076	1.345	1.761	2.145	2.624	2.977	3.787
15	0.866	1.074	1.341	1.753	2.131	2.602	2.947	3.733
16	0.865	1.071	1.337	1.746	2.120	2.583	2.921	3.686
17	0.863	1.069	1.333	1.740	2.110	2.567	2.898	3.646
18	0.862	1.067	1.330	1.734	2.101	2.552	2.878	3.610
19	0.861	1.066	1.328	1.729	2.093	2.539	2.861	3.579
20	0.860	1.064	1.325	1.725	2.086	2.528	2.845	3.552
21	0.859	1.063	1.323	1.721	2.080	2.518	2.831	3.527
22	0.858	1.061	1.321	1.717	2.074	2.508	2.819	3.505
23	0.858	1.060	1.319	1.714	2.069	2.500	2.807	3.485
24	0.857	1.059	1.318	1.711	2.064	2.492	2.797	3.467
25	0.856	1.058	1.316	1.708	2.060	2.485	2.787	3.450
26	0.856	1.058	1.315	1.706	2.056	2.479	2.779	3.435
27	0.855	1.057	1.314	1.703	2.052	2.473	2.771	3.421
28	0.855	1.056	1.313	1.701	2.048	2.467	2.763	3.408
29	0.854	1.055	1.311	1.699	2.045	2.462	2.756	3.396
30	0.854	1.055	1.310	1.697	2.042	2.457	2.750	3.385
40	0.851	1.050	1.303	1.684	2.021	2.423	2.704	3.307
50	0.849	1.047	1.299	1.676	2.009	2.403	2.678	3.261
60	0.848	1.045	1.296	1.671	2.000	2.390	2.660	3.232
70	0.847	1.044	1.294	1.667	1.994	2.381	2.648	3.211
80	0.846	1.043	1.292	1.664	1.990	2.374	2.639	3.195
100	0.845	1.042	1.290	1.660	1.984	2.364	2.626	3.174

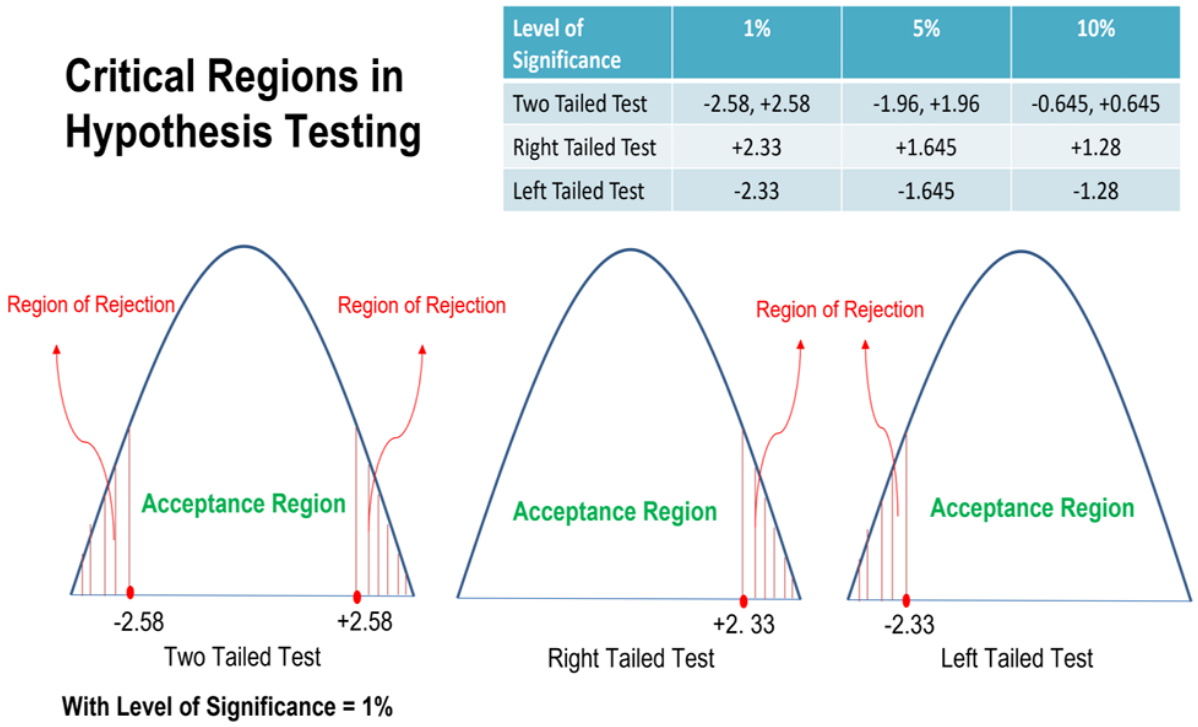
t Table

cum. prob	$t_{.50}$	$t_{.75}$	$t_{.80}$	$t_{.85}$	$t_{.90}$	$t_{.95}$	$t_{.975}$	$t_{.99}$	$t_{.995}$	$t_{.999}$	$t_{.9995}$
one-tail	0.50	0.25	0.20	0.15	0.10	0.05	0.025	0.01	0.005	0.001	0.0005
two-tails	1.00	0.50	0.40	0.30	0.20	0.10	0.05	0.02	0.01	0.002	0.001
df											
1	0.000	1.000	1.376	1.963	3.078	6.314	12.71	31.82	63.66	318.31	636.62
2	0.000	0.816	1.061	1.386	1.886	2.920	4.303	6.965	9.925	22.327	31.599
3	0.000	0.765	0.978	1.250	1.638	2.353	3.182	4.541	5.841	10.215	12.924
4	0.000	0.741	0.941	1.190	1.533	2.132	2.776	3.747	4.604	7.173	8.610
5	0.000	0.727	0.920	1.156	1.476	2.015	2.571	3.365	4.032	5.893	6.869
6	0.000	0.718	0.906	1.134	1.440	1.943	2.447	3.143	3.707	5.208	5.959
7	0.000	0.711	0.896	1.119	1.415	1.895	2.365	2.998	3.499	4.785	5.408
8	0.000	0.706	0.889	1.108	1.397	1.860	2.306	2.896	3.355	4.501	5.041
9	0.000	0.703	0.883	1.100	1.383	1.833	2.262	2.821	3.250	4.297	4.781
10	0.000	0.700	0.879	1.093	1.372	1.812	2.228	2.764	3.169	4.144	4.587
11	0.000	0.697	0.876	1.088	1.363	1.796	2.201	2.718	3.106	4.025	4.437
12	0.000	0.695	0.873	1.083	1.356	1.782	2.179	2.681	3.055	3.930	4.318
13	0.000	0.694	0.870	1.079	1.350	1.771	2.160	2.650	3.012	3.852	4.221
14	0.000	0.692	0.868	1.076	1.345	1.761	2.145	2.624	2.977	3.787	4.140
15	0.000	0.691	0.866	1.074	1.341	1.753	2.131	2.602	2.947	3.733	4.073
16	0.000	0.690	0.865	1.071	1.337	1.746	2.120	2.583	2.921	3.686	4.015
17	0.000	0.689	0.863	1.069	1.333	1.740	2.110	2.567	2.898	3.646	3.965
18	0.000	0.688	0.862	1.067	1.330	1.734	2.101	2.552	2.878	3.610	3.922
19	0.000	0.688	0.861	1.066	1.328	1.729	2.093	2.539	2.861	3.579	3.883
20	0.000	0.687	0.860	1.064	1.325	1.725	2.086	2.528	2.845	3.552	3.850
21	0.000	0.686	0.859	1.063	1.323	1.721	2.080	2.518	2.831	3.527	3.819
22	0.000	0.686	0.858	1.061	1.321	1.717	2.074	2.508	2.819	3.505	3.792
23	0.000	0.685	0.858	1.060	1.319	1.714	2.069	2.500	2.807	3.485	3.768
24	0.000	0.685	0.857	1.059	1.318	1.711	2.064	2.492	2.797	3.467	3.745
25	0.000	0.684	0.856	1.058	1.316	1.708	2.060	2.485	2.787	3.450	3.725
26	0.000	0.684	0.856	1.058	1.315	1.706	2.056	2.479	2.779	3.435	3.707
27	0.000	0.684	0.855	1.057	1.314	1.703	2.052	2.473	2.771	3.421	3.690
28	0.000	0.683	0.855	1.056	1.313	1.701	2.048	2.467	2.763	3.408	3.674
29	0.000	0.683	0.854	1.055	1.311	1.699	2.045	2.462	2.756	3.396	3.659
30	0.000	0.683	0.854	1.055	1.310	1.697	2.042	2.457	2.750	3.385	3.646
40	0.000	0.681	0.851	1.050	1.303	1.684	2.021	2.423	2.704	3.307	3.551
60	0.000	0.679	0.848	1.045	1.296	1.671	2.000	2.390	2.660	3.232	3.460
80	0.000	0.678	0.846	1.043	1.292	1.664	1.990	2.374	2.639	3.195	3.416
100	0.000	0.677	0.845	1.042	1.290	1.660	1.984	2.364	2.626	3.174	3.390
1000	0.000	0.675	0.842	1.037	1.282	1.646	1.962	2.330	2.581	3.098	3.300
z	0.000	0.674	0.842	1.036	1.282	1.645	1.960	2.326	2.576	3.090	3.291
	0%	50%	60%	70%	80%	90%	95%	98%	99%	99.8%	99.9%
					Confidence Level						

Question wording	Alternative hypothesis	Test type
Significantly **different** from	(H_1: mu ≠_0)	Two-tailed
Significantly **greater** than / **increases**	(H_1: mu > 0)	Right-tailed
Significantly **less** than / **decreases**	(H_1: mu< 0)	Left-tailed

Critical Regions

In hypothesis testing, critical region is represented by set of values, where null hypothesis is rejected. So it is also know as region of rejection. It takes different boundary values for different level of significance. Below info graphics shows the region of rejection that is critical region and region of acceptance with respect to the level of significance 1%.



Critical regions in Hypothesis Testing

Program:

#import necessary libraries

import numpy as np

from scipy import stats

import matplotlib.pyplot as plt

Given data

n = 20

sample_mean = 8.0

```
sample_sd = 2.0

# Hypothesized mean

mu0 = 0.0

alpha = 0.05

# Degrees of freedom

df = n - 1

# Standard Error

se = sample_sd / np.sqrt(n)

# t-statistic

t_stat = (sample_mean - mu0) / se

# Two-tailed p-value

p_value = 2 * stats.t.sf(abs(t_stat), df)

# Critical t-value

t_critical = stats.t.ppf(1 - alpha / 2, df)

# 95% Confidence Interval

ci_lower = sample_mean - t_critical * se

ci_upper = sample_mean + t_critical * se


# Display results

print("One-Sample t-Test")

print("-" * 30)

print(f"Sample Size (n)          : {n}")

print(f"Sample Mean              : {sample_mean:.2f} bpm")

print(f"Sample Standard Deviation: {sample_sd:.2f} bpm")

print(f"Degrees of Freedom       : {df}")

print(f"Standard Error           : {se:.4f}")
```

```
print(f"t-statistic          : {t_stat:.4f}")

print(f"Critical t-value     : ±{t_critical:.4f}")

print(f"Two-tailed p-value   : {p_value:.4e}")

print(f"95% Confidence Interval : ({ci_lower:.4f}, {ci_upper:.4f}) bpm")
```

Output:

```
One-Sample t-Test
------------------------------
Sample Size (n)          : 20
Sample Mean              : 8.00 bpm
Sample Standard Deviation: 2.00 bpm
Degrees of Freedom       : 19
Standard Error           : 0.4472
t-statistic              : 17.8885
Critical t-value         : ±2.0930
Two-tailed p-value       : 2.3956e-13
95% Confidence Interval  : (7.0640, 8.9360) bpm
```

Decision

```
print("\nDecision:")

if p_value < alpha:
    print("Reject the null hypothesis (H0).")
    print("Conclusion: The drug significantly changes the mean heart rate.")
else:
    print("Fail to reject the null hypothesis (H0).")
    print("Conclusion: There is insufficient evidence that the drug changes the mean heart rate.")
```

Output:

Decision:

Reject the null hypothesis (H0).

Conclusion: The drug significantly changes the mean heart rate.

Plot the t-distribution

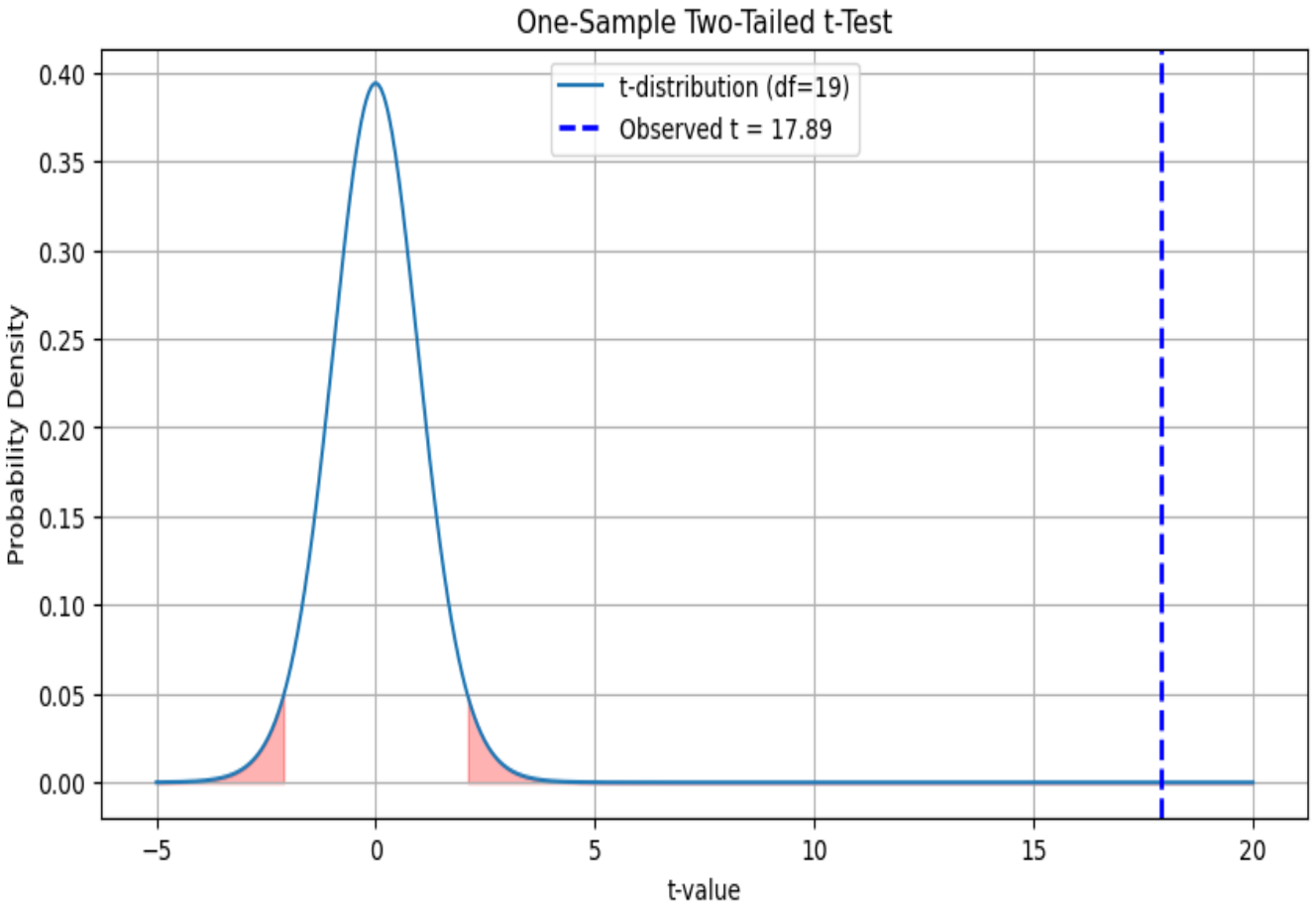
```
x = np.linspace(-5, 20, 1000)
y = stats.t.pdf(x, df)

plt.figure(figsize=(10, 5))
```

```
plt.plot(x, y, label=f"t-distribution (df={df})")

# Shade rejection regions
plt.fill_between(x, 0, y, where=(x <= -t_critical), color='red', alpha=0.3)
plt.fill_between(x, 0, y, where=(x >= t_critical), color='red', alpha=0.3)

# Plot observed t-statistic
plt.axvline(t_stat, color='blue', linestyle='--', linewidth=2,label=f"Observed t = {t_stat:.2f}")
plt.title("One-Sample Two-Tailed t-Test")
plt.xlabel("t-value")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)
plt.show()
```



Outcome of the Experiment:

EXPERIMENT 6:

A company is testing two versions of a webpage (A and B) to determine which version leads to more sales. Version A was shown to 1,000 users and resulted in 120 sales. Version B was shown to 1,200 users and resulted in 150 sales. Perform an A/B test to determine if there is a statistically significant difference in the conversion rates between the two versions. Use a 5% significance level.

A company is testing two versions of a webpage (A and B) to determine which version leads to more sales. Version A was shown to 1,000 users and resulted in 120 sales. Version B was shown to 1,200 users and resulted in 150 sales. Perform an A/B test to determine if there is a statistically significant difference in the conversion rates between the two versions. Use a 5% significance level.

For this problem, the **Z-test for two proportions** is the appropriate choice, **not a t-test**.

Why a Z-test?

The data consists of **counts of successes and failures**, not measurements of a continuous variable.

- Version A:
  - Users = 1000
  - Sales = 120
  - Conversion rate = 120/1000=0.12
- Version B:
  - Users = 1200
  - Sales = 150
  - Conversion rate = 150/1200=0.125

Here, each user either:

- **Made a purchase (Success = 1)**, or
- **Did not make a purchase (Failure = 0)**

So the outcome is **binary (categorical)**, and the goal is to compare **two population proportions**.

Why not a t-test?

A **t-test** is used to compare **means of continuous numerical data**.

Examples where a t-test is appropriate:

- Compare average heart rate between two groups.

- Compare average exam scores.
- Compare average blood pressure.

Your A/B testing problem does **not** involve continuous measurements. It involves conversion rates (proportions), so a t-test is not appropriate.

Why is the Z-test valid here?

The sample sizes are large.

For Version A:

- np=1000×0.12=120np = 1000 \times 0.12 = 120np=1000×0.12=120
- n(1−p)=1000×0.88=880n(1-p) = 1000 \times 0.88 = 880n(1−p)=1000×0.88=880

For Version B:

- np=1200×0.125=150np = 1200 \times 0.125 = 150np=1200×0.125=150
- n(1−p)=1200×0.875=1050n(1-p) = 1200 \times 0.875 = 1050n(1−p)=1200×0.875=1050

Since all these values are much greater than 5 (and also greater than 10), the sampling distribution of the difference in proportions is approximately normal. Therefore, the **Z-test for two proportions** is appropriate.

Hypotheses

H0: $p_A = p_B$ There is **no difference** in conversion rates.
H1: $p_A \neq p_B$ There **is a difference** in conversion rates.

Because the question asks whether there is a **statistically significant difference**, this is a **two-tailed test**.

Rule of thumb

- **Use a t-test** when comparing **means** of continuous numerical data (e.g., average heart rate, average marks, average weight).
- **Use a Z-test for proportions** when comparing **percentages or conversion rates** (e.g., click-through rates, purchase rates, pass rates, defect rates).

To determine whether there's a statistically significant difference between the conversion rates of versions A and B, we can perform a two-proportion z-test.

Step-by-step Approach:

Define Hypotheses: Null hypothesis (H_0): $p_1 = p_2$ (no difference in conversion rates)

Alternative hypothesis (H_1): $p_1 \neq p_2$ (two-tailed test)

Where:

p_1 = conversion rate of version A

p_2 = conversion rate of version B

Given Data

Version A: $n_1 = 1000$ $x_1 = 120$

Version B: $n_2 = 1200$ $x_2 = 150$

Test Statistic (Z-test)

$$p_1 = \frac{x_1}{n_1}$$

$$p_2 = \frac{x_2}{n_2}$$

$$p = \frac{x_1 + x_2}{n_1 + n_2}$$

$$SE = \sqrt{p(1-p)\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}$$

$$Z = \frac{p_1 - p_2}{SE}$$

Program:

```
#import necessary libraries
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Given data
n = 20
sample_mean = 8.0
sample_sd = 2.0
mu0 = 0.0          # Hypothesized mean
alpha = 0.05

# Degrees of freedom
df = n - 1
# Standard Error
se = sample_sd / np.sqrt(n)
# t-statistic
t_stat = (sample_mean - mu0) / se
# Two-tailed p-value
p_value = 2 * stats.t.sf(abs(t_stat), df)
# Critical t-value
t_critical = stats.t.ppf(1 - alpha / 2, df)
# 95% Confidence Interval
ci_lower = sample_mean - t_critical * se
ci_upper = sample_mean + t_critical * se

# Display results
print("One-Sample t-Test")
print("-" * 30)
print(f"Sample Size (n)         : {n}")
print(f"Sample Mean             : {sample_mean:.2f} bpm")
print(f"Sample Standard Deviation: {sample_sd:.2f} bpm")
print(f"Degrees of Freedom      : {df}")
```

```
print(f"Standard Error          : {se:.4f}")
print(f"t-statistic             : {t_stat:.4f}")
print(f"Critical t-value        : ±{t_critical:.4f}")
print(f"Two-tailed p-value      : {p_value:.4e}")
print(f"95% Confidence Interval : ({ci_lower:.4f}, {ci_upper:.4f}) bpm")
```

Output:

```
One-Sample t-Test
------------------------------
Sample Size (n)           : 20
Sample Mean              : 8.00 bpm
Sample Standard Deviation: 2.00 bpm
Degrees of Freedom       : 19
Standard Error           : 0.4472
t-statistic              : 17.8885
Critical t-value         : ±2.0930
Two-tailed p-value       : 2.3956e-13
95% Confidence Interval : (7.0640, 8.9360) bpm
```

Decision

```
print("\nDecision:")
if p_value < alpha:
    print("Reject the null hypothesis (H0).")
    print("Conclusion: The drug significantly changes the mean heart rate.")
else:
    print("Fail to reject the null hypothesis (H0).")
    print("Conclusion: There is insufficient evidence that the drug changes the mean heart rate.")
```

Output:

Decision:

Reject the null hypothesis (H0).

Conclusion: The drug significantly changes the mean heart rate.

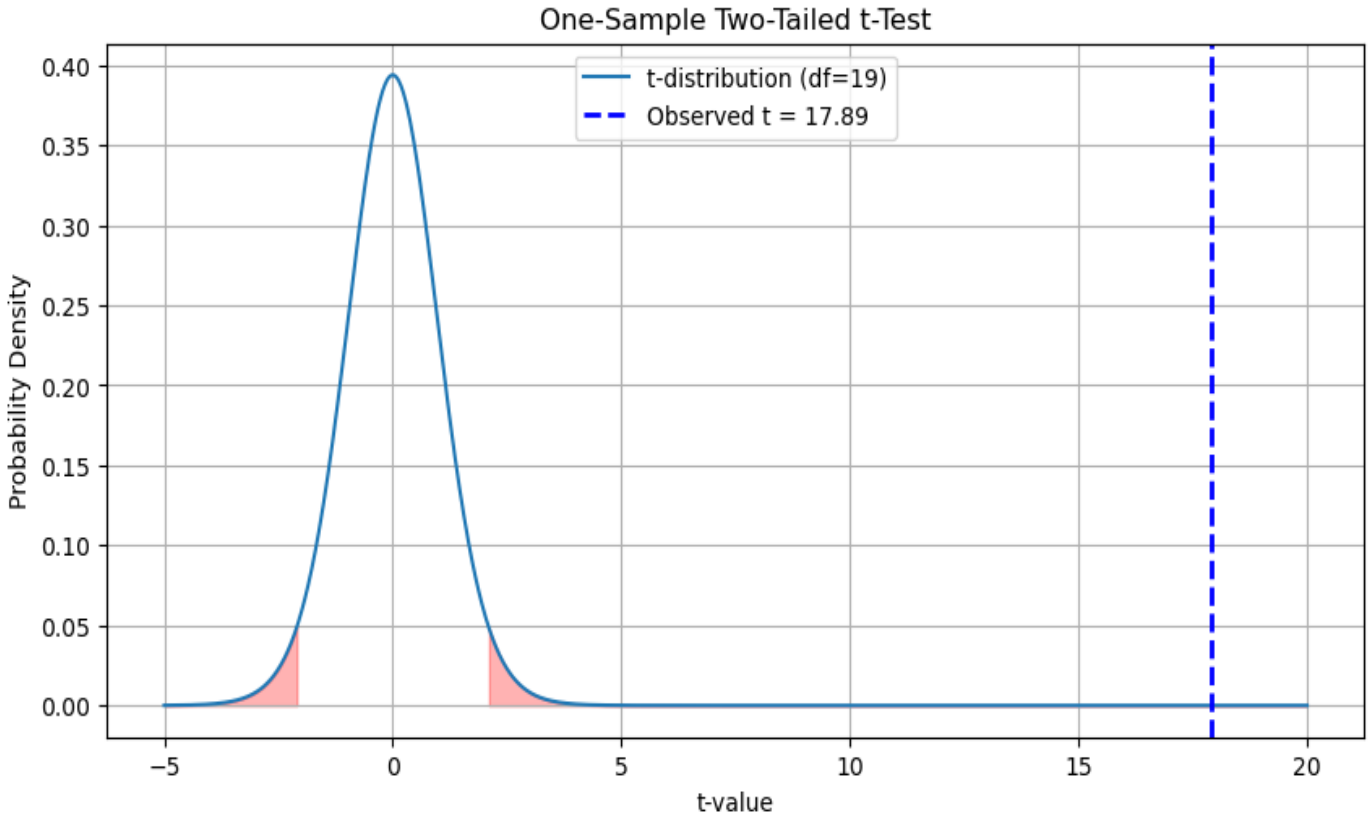
Plot the t-distribution

```
x = np.linspace(-5, 20, 1000)
y = stats.t.pdf(x, df)
plt.figure(figsize=(10, 5))
plt.plot(x, y, label=f"t-distribution (df={df})")
```

```
# Shade rejection regions
plt.fill_between(x, 0, y, where=(x <= -t_critical), color='red', alpha=0.3)
plt.fill_between(x, 0, y, where=(x >= t_critical), color='red', alpha=0.3)

# Plot observed t-statistic
plt.axvline(t_stat, color='blue', linestyle='--', linewidth=2,label=f"Observed t = {t_stat:.2f}")
plt.title("One-Sample Two-Tailed t-Test")
plt.xlabel("t-value")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Outcome of the Experiment:

EXPERIMENT -7

You are comparing the average daily sales between two stores. Store A has a mean daily sales value of 1, 000 *with a standard deviation of* 100 over 30 days, and Store B has a mean daily sales value of 950 *with a standard deviation of* 120 over 30 days. Conduct a two-sample t-test to determine if there is a significant difference between the average sales of the two stores at the 5% significance level.

To compare the average daily sales between Store A and Store B, we will conduct an independent two - sample t-test, assuming the samples are independent and approximately normally distributed.

Step-by-Step Hypothesis Test

Define Hypotheses: Null hypothesis (H_0): $\mu_1 = \mu_2$ (no difference in average sales)

Alternative hypothesis (H_1): $\mu_1 \neq \mu_2$ (two-tailed test) Where:

μ_1 = mean sales of Store A μ_2 = mean sales of Store B

Given Data: Store	Mean ()	*StdDev*()	n (days)
Store A	1,000	100	30
Store B	950	120	30

Use an independent two-sample t-test

$$t = \frac{(\overline{x_1} - \overline{x_2}) - (\mu_1 - \mu_2)}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

$$df = \frac{\left(\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}\right)^2}{\frac{1}{n_1 - 1}\left(\frac{S_1^2}{n_1}\right)^2 + \frac{1}{n_2 - 1}\left(\frac{S_2^2}{n_2}\right)^2}$$

Program:

```
#import necessary libraries
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

# Given data
n1 = 30
n2 = 30
mean1 = 1000      # Store A mean sales
mean2 = 950       # Store B mean sales
```

```
sd1 = 100        # Store A standard deviation
sd2 = 120        # Store B standard deviation
alpha = 0.05

# Standard Error
se = np.sqrt((sd1**2 / n1) + (sd2**2 / n2))
# t-statistic
t_stat = (mean1 - mean2) / se
# Welch's degrees of freedom
df = ((sd1**2 / n1 + sd2**2 / n2)**2) / (
   ((sd1**2 / n1)**2 / (n1 - 1)) +
   ((sd2**2 / n2)**2 / (n2 - 1))
)

# Two-tailed p-value
p_value = 2 * stats.t.sf(abs(t_stat), df)
# Critical t-value
t_critical = stats.t.ppf(1 - alpha/2, df)
# 95% Confidence Interval
difference = mean1 - mean2
ci_lower = difference - t_critical * se
ci_upper = difference + t_critical * se

# Display Results
print("Independent Two-Sample t-Test")
print("-" * 40)
print(f"Store A Mean Sales      : ${mean1:.2f}")
print(f"Store B Mean Sales      : ${mean2:.2f}")
print(f"Mean Difference         : ${difference:.2f}")
print(f"Standard Error          : {se:.4f}")
print(f"Degrees of Freedom      : {df:.2f}")
print(f"t-statistic             : {t_stat:.4f}")
print(f"Critical t-value        : ±{t_critical:.4f}")
print(f"Two-tailed p-value      : {p_value:.4f}")
```

```
print(f"95% Confidence Interval : (${ci_lower:.2f}, ${ci_upper:.2f})")
```

Output:

```
Independent Two-Sample t-Test
----------------------------------------
Store A Mean Sales      : $1000.00
Store B Mean Sales      : $950.00
Mean Difference         : $50.00
Standard Error          : 28.5190
Degrees of Freedom      : 56.17
t-statistic             : 1.7532
Critical t-value        : ±2.0031
Two-tailed p-value      : 0.0850
95% Confidence Interval : ($-7.13, $107.13)
```

Decision

```
print("\nDecision:")
if p_value < alpha:
    print("Reject the null hypothesis (H0).")
    print("Conclusion: There is a significant difference in average daily sales.")
else:
    print("Fail to reject the null hypothesis (H0).")
    print("Conclusion: There is no significant difference in average daily sales.")
```

Output:

Decision:

Fail to reject the null hypothesis (H0).

Conclusion: There is no significant difference in average daily sales.

Plot

```
x = np.linspace(-4, 4, 1000)
y = stats.t.pdf(x, df)
plt.figure(figsize=(10,5))
plt.plot(x, y, label=f"t-distribution (df={df:.1f})")
```

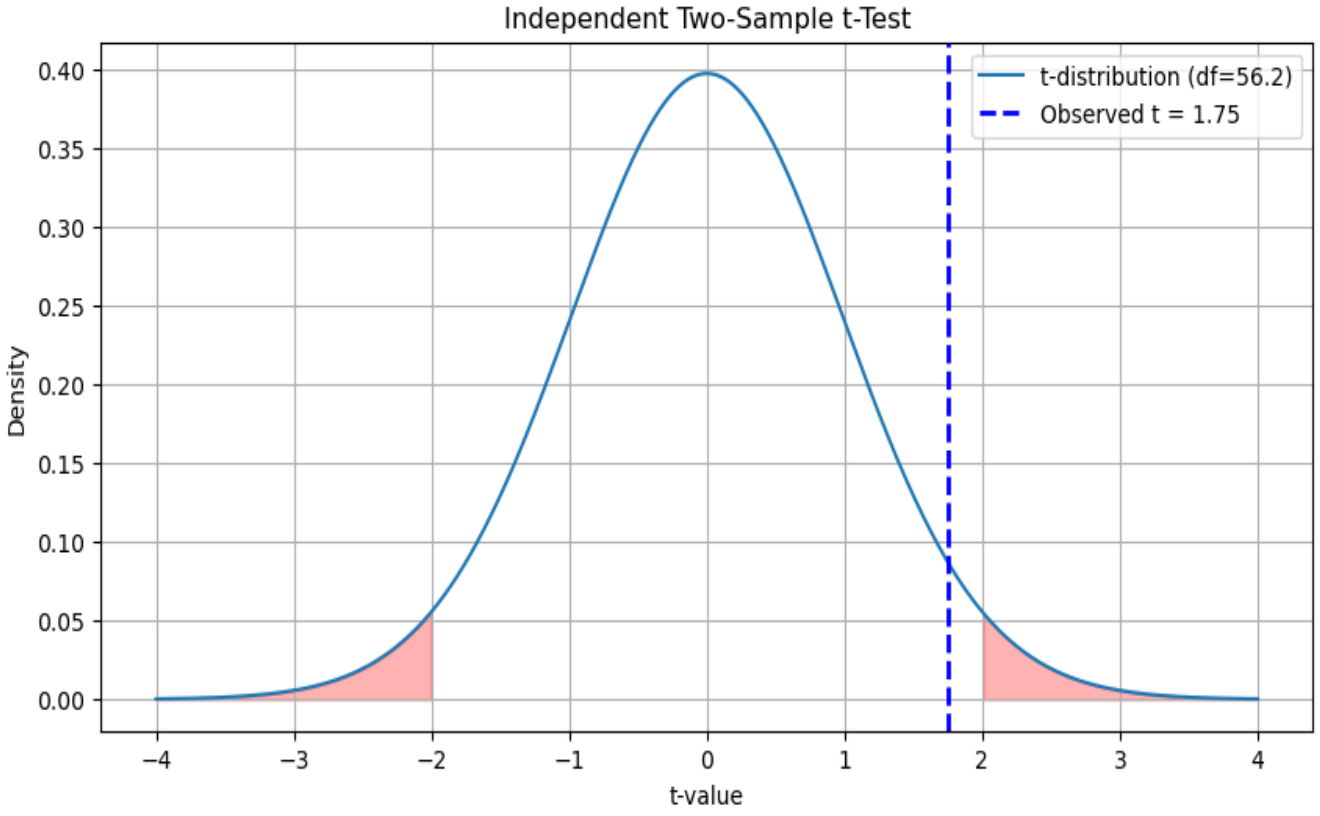
Rejection regions

```
plt.fill_between(x, 0, y, where=(x <= -t_critical),color='red', alpha=0.3)
```

```
plt.fill_between(x, 0, y, where=(x >= t_critical), color='red', alpha=0.3)

# Observed t
plt.axvline(t_stat, color='blue', linestyle='--',linewidth=2, label=f'Observed t = {t_stat:.2f}')
plt.title("Independent Two-Sample t-Test")
plt.xlabel("t-value")
plt.ylabel("Density")
plt.grid(True)
plt.legend()
plt.show()
```

Output:



Outcome of the Experiment:

EXPERIMENT-8

A company collects data on employees' salaries and records their education level as a categorical variable with three levels: "High School", "Bachelor's", and "Master's". Fit a multiple linear regression model to predict salary using education level (as a factor variable) and years of experience. Interpret the coefficients for the education levels in the regression model.

Import necessary libraries

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df=pd.read_csv('synthetic_salary_data.csv')
df.head()
```

Number	Experience (Years)	Qualification Level	Salary
0	6	3	52676
1	19	3	81949
2	28	4	104930
3	14	4	69120
4	10	3	58475

```
# Splitting features (X) and target variable (y)
X = df[['Qualification Level', 'Experience (Years)']]
# Independent variables
y = df['Salary']  # Dependent variable
```

Splitting the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Creating and training the model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Making predictions

y_pred = model.predict(X_test)

Evaluating the model

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

Displaying results

print("Model Coefficients:", model.coef_)

print("Model Intercept:", model.intercept_)

print("Mean Squared Error:", mse)

print("R-squared Score:", r2)

Predicting salary for a new input

new_data = np.array([[6, 3]]) # Example: 6 years of experience, Master's degree

predicted_salary = model.predict(new_data)

print("Predicted Salary for new data:", predicted_salary[0])

Model Coefficients: [4935.68791915 2010.608594]

Model Intercept: 25065.425408432246

Mean Squared Error: 2315293.8215379473

R-squared Score: 0.9943910870761583

Predicted Salary for new data: 60711.37870533018

----- Step 3: Visualization -----

sns.set(style="whitegrid")

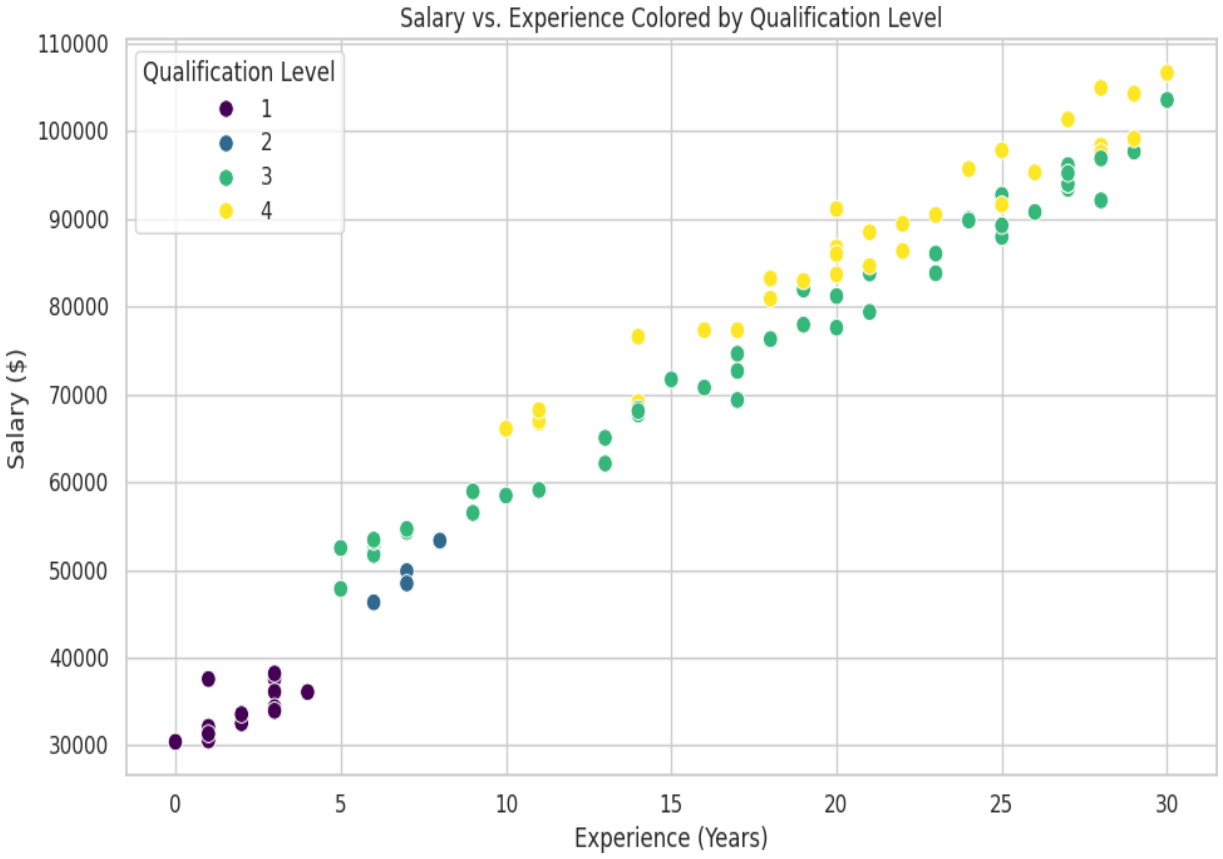
Salary vs Experience

plt.figure(figsize=(10, 6))

```
sns.scatterplot(data=df, x='Experience (Years)', y='Salary', hue='Qualification Level', palette='viridis', s=70)
```

plt.title("Salary vs. Experience Colored by Qualification Level")

```
plt.xlabel("ExperienceYears)")
plt.ylabel("Salary ($)")
plt.legend(title="Qualification          Level")
plt.tight_layout()
plt.show()
```



Outcome of the Experiment:

EXPERIMENT-9

You have data on housing prices and square footage and notice that the relationship between square footage and price is nonlinear. Fit a spline regression model to allow the relationship between square footage and price to change at 2,000 square feet. Explain how spline regression can capture different behaviours of the relationship before and after 2,000 square feet.

What Is Spline Regression?

Spline regression is a regression technique that models nonlinear relationships using piecewise polynomial functions joined together at specific points called knots.

So, instead of making one equation to describe the entire relationship, spline regression breaks the predictor range into smaller pieces and fits a separate polynomial on each piece. These pieces meet at the knots, and constraints make sure the transitions are smooth.

The end result is somewhere between two extremes. It's more flexible than linear regression, which can only make a single straight line through your data. And it's more structured than fully unconstrained nonlinear models like deep neural networks or kernel methods, which can fit almost anything but tell you very little about what they've fit.

Why Spline Regression Is Needed

Linear regression is the default starting point for modeling relationships, but it has a strong assumption: the effect of a predictor on the outcome stays constant across the range. When the actual relationship curves or changes direction, a straight line will underfit it. You get errors at the extremes and a model that can’t follow the pattern.

The fix is to use a more flexible model. High-degree polynomial regression is one option - you add x^2, x^3, x^4 terms until the curve bends enough to match your data. But polynomials become unstable at the edges of the data as they swing up or down where you have few data points. That behavior is called Runge's phenomenon, and it makes high-degree polynomials risky for prediction.

Spline regression is between these two extremes.

You get local flexibility where the data bends, without the global instability of a single high-degree polynomial. Each segment is a low-degree polynomial (usually cubic) so no single part can behave unexpectedly. And because the segments are joined smoothly at the knots, the overall curve still looks like one continuous function.

So, in a nutshell, a spline offers enough flexibility to follow complex patterns and enough structure to stay behave at extremes.

How Spline Regression Works

Spline regression always follows a simple three-step workflow.

1. Divide the predictor range into regions: You pick a set of knots along the predictor axis. These knots split the range into intervals. If you place three knots, you get four regions. The knots are the boundaries between segments.

2. Fit a polynomial within each region: Inside each interval, the model fits a low-degree polynomial. Each region gets its own polynomial coefficients, so the curve can bend differently in different parts of the predictor range. A region where the relationship is nearly flat gets a near-flat polynomial. A region where the relationship curves sharply gets a more curved one.

3. Connect the regions: The model enforces continuity constraints at every knot. The polynomial values from both sides must match at the knot, so the curve has no jumps. For cubic splines, the first and second derivatives must also match, which means no sharp corners and no abrupt changes in curvature.

Spline Regression in Machine Learning and Statistics

Spline regression shows up wherever someone needs to model a smooth nonlinear effect without committing to a specific functional form. Here are a couple of common areas:

- Time series trends: A common use is separating the smooth trend in a time series from short-term fluctuations. A spline fit on time as the predictor gives you a flexible trend line that adapts to changes in direction without overreacting to noise. Economists and analysts use this all the time when they want to describe the underlying trajectory of a variable - think GDP or stock prices - without committing to a linear or exponential assumption.

- Economics and econometrics: Splines are used to model relationships where the effect of a variable changes over its range. The effect of income on consumption isn't constant, as low-income households spend differently than high-income ones. A spline sees this without assuming a specific functional form.

- Healthcare and biostatistics: Many health outcomes have nonlinear relationships with predictors like age, BMI, blood pressure, or biomarker levels. Risk of disease often follows a U-shape or J-shape - both extremes are dangerous, the middle is safe. A linear model would miss this. Cubic

splines and natural cubic splines are the standard tool for modeling these effects, and they're built into most statistical software used in clinical research.

- Environmental and ecological modeling: Splines also show up when modeling how species or climate variables respond to environmental gradients. Things like temperature, elevation, and rainfall often have nonlinear effects on outcomes like species abundance or crop yield. Splines let researchers fit the response curve without specifying its shape beforehand.

Advantages of Spline Regression

Here's a couple of advantages of splines when compared to more popular machine learning models:

- Models nonlinear relationships without knowing the form: You don't have to commit to a specific functional shape. Splines let the data shape the curve, so you can model bends, plateaus, and direction changes without knowing in advance where they'll occur.
- Smooth and interpretable fits: The output is a continuous curve, not a black-box prediction. You can plot the spline, see how the response changes across the predictor range, and explain it to a non-technical audience.
- More stable than high-degree polynomials: Each segment is a low-degree polynomial, so no single piece can behave widely at extremes. Boundary behavior is much more controlled, especially with natural cubic splines.
- Works into existing regression workflows: Splines work inside standard linear regression. You can combine them with other predictors, regularization, mixed effects, and any other tool that fits on top of OLS.

Limitations of Spline Regression

Like most models, splines come with a couple tradeoffs that you have to know about:

- Knot selection is the hard part: Picking the number and location of knots takes work. Defaults are good sometimes, but in many cases you'll need cross-validation or domain knowledge to get a good fit. There's no one rule that always works.
- Interpretation gets harder with more knots: A spline with three knots is simple enough to interpret. A spline with twenty knots isn't. You can still plot the curve, but explaining what the model is doing in any rigorous way becomes much harder.

Program:

#Read dataset

df=pd.read_csv('housing_prices_spline1.csv')

df.head()

No.	*SquareFootage*	*Price*
0	*500*	*129967*
1	*530*	*128163*
2	*561*	*140568*
3	*591*	*153867*
4	*621*	*140840*

import statsmodels.api as sm

from patsy import dmatrix

Create spline basis with a knot at 2000

spline_basis = dmatrix("bs(SquareFootage, knots=(2000,), degree=1, include_intercept=False)",

{"SquareFootage": df['SquareFootage']}, return_type='dataframe')

Fit linear regression model on spline basis

model = sm.OLS(df['Price'], spline_basis).fit()

print(model.summary())

Predict fitted values

df['FittedPrice'] = model.predict(spline_basis)

OLS Regression Results

Dep. Variable:	Price	R-squared:	0.993
Model:	OLS	Adj. R-squared:	0.993
Method:	Least Squares	F-statistic:	7251.
Date:	Wed, 30 Jul 2025	Prob (F-statistic):	2.45e-106
Time:	10:45:00	Log-Likelihood:	-1052.6
No. Observations:	100	AIC:	2111.
Df Residuals:	97	BIC:	2119.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.24e+05	2391.321	51.837	0.000	1.19e+05	1.29e+05
bs(SquareFootage, knots=(2000,), degree=1, include_intercept=False)[0]	2.243e+05	3510.735	63.890	0.000	2.17e+05	2.31e+05
bs(SquareFootage, knots=(2000,), degree=1, include_intercept=False)[1]	3.764e+05	3139.800	119.875	0.000	3.7e+05	3.83e+05

Omnibus:	0.471	Durbin-Watson:	2.046
Prob(Omnibus):	0.790	Jarque-Bera (JB):	0.483
Skew:	-0.158	Prob(JB):	0.785
Kurtosis:	2.875	Cond. No.	5.78

Notes: *Standard Errors assume that the covariance matrix of the errors is correctly specified.*

```
import matplotlib.pyplot as plt

 plt.figure(figsize=(10, 6))

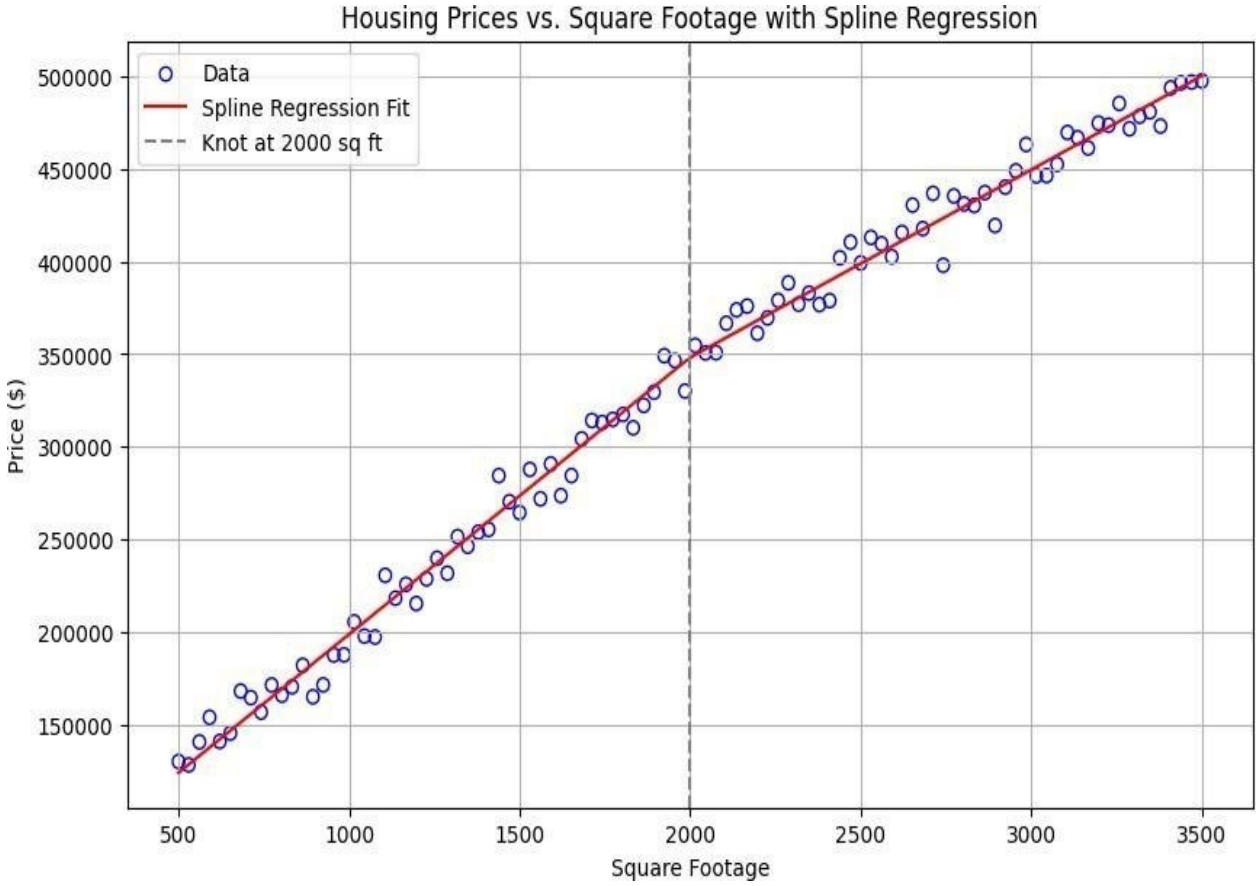
# Scatter plot of actual data points

plt.scatter(df['SquareFootage'], df['Price'], facecolors='none', edgecolors='blue', label='Data')

# Plot the fitted spline regression line

plt.plot(df['SquareFootage'], df['FittedPrice'], color='red', label='Spline Regression Fit')

# Mark the knot at 2000 sq ft
plt.axvline(x=2000, color='grey', linestyle='--', label='Knot at 2000 sq ft')
plt.xlabel('Square Footage')
plt.ylabel('Price ($)')
plt.title('Housing Prices vs. Square Footage with Spline Regression')
plt.legend()
plt.grid(True)
plt.show()
```


Housing Prices vs. Square Footage with Spline Regression
Data
Spline Regression Fit
Knot at 2000 sq ft
Price ($)
Square Footage
500000
450000
400000
350000
300000
250000
200000
150000
500
1000
1500
2000
2500
3000
3500

Outcome of the Experiment:

EXPERIMENT- 10:

A hospital is using a Poisson regression model (a type of GLM) to predict the number of emergency room visits per week based on patient age and medical history. The model is given by: *Log(λ) =2.5-0.03*Age+0.5*condition* where λ is the expected number of visits per week, Age is the patient's age, and condition is a binary variable (1 if the patient has a chronic condition, 0 otherwise). Interpret the coefficients of Age and condition. What is the expected number of visits per week for a 60- year-old patient with a chronic condition? How would the expected number of visits change if the patient did not have a chronic condition?

Given Poisson Regression Model:

$\log(\lambda)=2.5-0.03*Age+0.5*conditio$

n Where:

- λ = expected number of emergency room (ER) visits per week.
- Age = patient's age in years.
- condition = 1 if patient has a chronic condition, 0 otherwise.

Interpretation of Coefficients:

1. **Age coefficient (-0.03)**:
   - For each **additional year of age**, the log of expected ER visits **decreases by 0.03**.
   - In terms of rate: $e^{-0.03}\approx 0.9704$

     So, each additional year of age is associated with a **~3% decrease** in the expected number of ER visits.

2. **Condition coefficient (0.5)**:
   - Having a chronic condition increases the **log of expected ER visits by 0.5**.
   - In terms of rate: $e^{0.5}\approx 1.65$

     So, patients with a chronic condition have **~65% more expected ER visits** than those without, all else equal.

Expected Visits for 60-Year-Old with Chronic Condition:
Plug in:

- Age = 60
- condition = 1

$\log(\lambda)=2.5-0.03*60+0.5*1=2.5-1.8+0.5=1.$

2 $\lambda=e^{1.2}\approx 3.32$

Expected ER visits per week = ~3.32

If the Patient Did NOT Have a Chronic Condition:

Same age (60), but condition = 0:

$\log(\lambda)=2.5-1.8+0=0.7$
$\lambda=e^{0.7}\approx 2.01$

Expected ER visits per week = ~2.01

Effect of the Chronic Condition:

$3.32/2.01\approx 1.65$

So, the chronic condition increases expected ER visits by **~65%**, consistent with the coefficient interpretation.

Program

```
import math

def expected_visits(age, condition):

log_lambda = 2.5 - 0.03 * age + 0.5 * condition lambda_val = math.exp(log_lambda)

return lambda_val

age = 60

visits_with_condition = expected_visits(age, condition=1)

visits_without_condition = expected_visits(age, condition=0)

print(f"Expected visits with chronic condition: {visits_with_condition:.2f}")

print(f"Expected visits without chronic condition: {visits_without_condition:.2f}")

print(f"Increase due to condition: {(visits_with_condition / visits_without_condition):.2f}x")
```

Expected visits with chronic condition: 3.32
Expected visits without chronic condition: 2.01
Increase due to condition: 1.6

Outcome of the Experiment:

EXPERIMENT -11

A bakery claims that its new cookie recipe is lower in calories compared to the old recipe, which had a mean calorie count of 200. You sample 40 new cookies and find a mean of 190 calories with a standard deviation of 15 calories. Perform a one-tailed t-test to determine if the new recipe has significantly fewer calories at a 5% significance level.

Hypotheses: Null Hypothesis (H_0): $\mu = 200$ (mean calories of new recipe is the same as old)

Alternative Hypothesis (H_1): $\mu < 200$ (mean calories of new recipe is less than old)

Given: Sample mean xbar = 190

Population mean under H_0 (μ_0) = 200

Sample standard deviation (s) = 15

Sample size (n) = 40

Significance level (α) = 0.05

T-test formula:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

$$t = \frac{190 - 200}{15/\sqrt{40}} \approx \frac{-10}{2.37} \approx -4.22$$

Program

```
from scipy import stats
import math

# Given values
mu_0 = 200     # Old recipe mean sample_mean =190
sample_std = 15
n = 40
alpha = 0.05

# Calculate t-statistic
t_stat = (sample_mean - mu_0) / (sample_std / math.sqrt(n))
```

```
# Degrees of freedom
df = n - 1

# Critical t-value for one-tailed test
t_critical = stats.t.ppf(alpha, df)

# p-value
p_value = stats.t.cdf(t_stat, df)

# Output results
print(f"t-statistic: {t_stat:.4f}")
print(f"Critical t-value (one-tailed, α=0.05): {t_critical:.4f}")
print(f"p-value: {p_value:.4f}")

if t_stat < t_critical:
    print("Reject tWhehantuclaln Ihhyeploptyhoeusibsu:ild?he new recipe has significantly fewer
        calories.")
else:
    print("Fail to reject the null hypothesis: No significant reduction in calories.")
```

t-statistic: -4.2164

Critical t-value (one-tailed, α=0.05): -1.6849 p-value: 0.0001

Reject the null hypothesis: The new recipe has significantly fewer calories.

Conclusion: If t-statistic < t-critical and p-value < 0.05, you reject the null.
In this case, since t ≈ -4.22, it's less than the critical value (~ -1.685), and the p-value is very small (<< 0.05), you can conclude:The new cookie recipe has significantly fewer calories at the 5% level.

Outcome of the Experiment